HYACINTH AI

Internal Security Assessment & Remediation Report

Mnemne — local-first document intelligence for macOS

A summary of Mnemne's internal penetration-testing program: the surfaces assessed, the issues identified, and the controls now in place. All identified findings have been remediated.

Product	Mnemne (macOS desktop application)
Assessment type	Internal security review / penetration testing
Document	Assessment & Remediation Report, v1.1
Period	Through June 2026
Stage	Pre-release (beta) — mitigations ship ahead of GA
Status	All identified findings remediated
Contact	security@hyacinth.ai

1. About this report

This document summarizes the results of Hyacinth AI's **internal** security-assessment and penetration-testing program for Mnemne. It is provided to give customers transparency into how the product is tested and hardened. It is an internal assessment performed by the engineering team — **not** an independent third-party audit; an external assessment is on the roadmap and will supersede this document when complete.

To avoid handing a roadmap to attackers, findings are described at the level of **category, impact, and remediation**. Reproducible exploit detail for fixed issues is deliberately omitted.

2. Scope & methodology

Mnemne is a local-first macOS application: a native shell, a local application server, a document-parsing sidecar, and an on-device inference engine, all bound to loopback. The assessment was threat-model driven, focusing on the realistic external attackers for a local-first desktop app: a malicious web page visited while the app runs, a malicious document the user indexes, and a network attacker targeting the update or OAuth channels.

The following attack surfaces were enumerated and assessed:

#	Surface	Result
1	Local API perimeter (loopback HTTP servers)	**Hardened**
2	Native IPC + capability allowlist	**Reviewed**
3	Webview content rendering (XSS bridge)	**Closed**
4	Custom-scheme / deep links	**Hardened**
5	OAuth + secret handling	**Hardened**
6	Auto-update integrity (supply chain)	**Verified**
7	Document parsers (untrusted input)	**Hardened + fuzzed**
8	Subprocess / shell-out surface	**Hardened**
9	Server-side request forgery (SSRF)	**Fixed**
10	Data at rest & runtime entitlements	Reviewed; roadmap items

3. Findings summary

Eight findings were identified and **all have been remediated**. Each ships with a regression test where applicable, so the fix cannot silently revert.

ID	Category	Severity	Status
A-01	Command execution via document-open path	High	Remediated
A-02	Missing origin/host validation on local API	High	Remediated
A-03	Server-side request forgery in web fetch	High	Remediated
A-04	Unbounded document-parser resource use	High	Remediated
A-05	Forgeable custom-scheme callback	Medium	Remediated
A-06	Secrets written to local diagnostic logs	Medium	Remediated
A-07	OAuth replay protection (verification + test)	Info	Verified
A-08	Loopback parser/AI servers lacked anti-rebinding guard	High	Remediated

4. Findings detail

A-01 — Command execution via document-open path · High

Issue: the route that opens an indexed document invoked the operating-system open handler through a shell, allowing a maliciously named file (or, combined with A-02, a hostile web page) to influence command execution.

Remediation: the route now invokes the open handler **without a shell** and only on validated targets (an existing absolute path, an `http(s)` URL, or a recognized cloud link). Input that isn't one of those shapes is rejected. Covered by unit tests pinning the safe-invocation and rejection behavior.

A-02 — Missing origin/host validation on the local API · High

Issue: the loopback application API accepted requests without validating their origin, exposing it to DNS-rebinding and cross-site request forgery from any web page the user visited while the app was running.

Remediation: a request-origin guard now rejects requests whose `Host` is not a loopback address (anti DNS-rebinding) and state-changing requests whose `Origin` is non-loopback (anti CSRF). Implemented as middleware over the entire API surface, with unit tests for each case.

A-03 — Server-side request forgery in web fetch · High

Issue: the optional web-search feature fetched result URLs without validating them, so an attacker-influenced result could direct a fetch at the app's own local services or at cloud-metadata/private addresses.

Remediation: a URL guard now permits only public `http(s)` endpoints — loopback, private, link-local, and cloud-metadata addresses are rejected, both for the URL itself (including DNS resolution) and for every redirect hop. Covered by unit tests.

A-04 — Unbounded document-parser resource use · High

Issue: the document-parsing pipeline — the largest untrusted-input surface — applied no size cap and no "zip-bomb" protection, so a crafted file could exhaust memory.

Remediation: size limits and a compression-ratio / uncompressed-size guard now run **before** any parser touches a file; Office-format XML parsing has external-entity resolution disabled (no XXE); and an adversarial-input fuzz corpus confirms the parsers handle malformed input gracefully. Per-parse time is also bounded.

A-05 — Forgeable custom-scheme callback · Medium

Issue: a sensitive `mnemne://` callback could be triggered by an externally-opened URL, letting a web page inject unsolicited cloud-file selections (a corpus-poisoning vector).

Remediation: deep links are now routed by source — the sensitive OAuth and cloud-picker callbacks are honored only from in-app navigations, never from externally-opened (forgeable) URLs. Covered by unit tests.

A-06 — Secrets written to local diagnostic logs · Medium

Issue: OAuth callback parameters (including the authorization code) were written to local diagnostic logs.

Remediation: callback query strings are redacted before logging, so authorization codes and related parameters no longer reach disk.

A-07 — OAuth replay protection · Informational

Finding: review confirmed the OAuth PKCE `state` is high-entropy, validated on callback, and consumed before token exchange (so a replayed callback is rejected). No change was required; a **replay regression test** was added so the single-use guarantee cannot silently regress.

A-08 — Loopback parser/AI servers lacked an anti-rebinding guard · High

Issue: the origin/host validation added in A-02 protected the main application API, but the two other loopback services — the document-parsing sidecar and the on-device inference engine — did not yet enforce the same `Host` check, leaving them reachable via a DNS-rebinding attack from a web page open while the app runs.

Remediation: the sidecar now installs a host-guard middleware that rejects any request whose `Host` is not a loopback address, so all three loopback services share the same anti-rebinding posture. Covered by unit tests (including bare-IPv6 loopback).

5. Additional hardening & surfaces closed

- **Webview content rendering:** document, model, and web-search content is rendered with HTML escaping (no raw HTML), and a Content-Security-Policy restricts script, object, and frame sources — closing the untrusted-content-to-script bridge.
- **Subprocess surface:** all parser shell-outs were confirmed to use argument arrays (no shell interpolation).
- **Document parsers:** a 13-case adversarial fuzz corpus exercises empty, garbage, truncated, and corrupt-container inputs.
- **Supply chain:** JavaScript, Python, and Rust dependencies are scanned for known vulnerabilities (`npm audit` + `pip-audit`) and a complete CycloneDX **Software Bill of Materials** covering all three ecosystems is generated and **published with every release** (`pub-mnemne/doc/sbom.json`); the publish pipeline aborts if a complete SBOM cannot be produced, so the published artifact can never silently drift from what ships.

6. Roadmap items

Tracked, lower-urgency hardening:

- App-level encryption of the local index beyond macOS FileVault.
- Tightening Hardened Runtime entitlements now that bundled libraries share one signing Team ID.
- A hardware-backed update-signing key and release-provenance attestation.
- An **independent third-party security assessment**, which will supersede this internal report.

7. Conclusion

Across ten enumerated attack surfaces, eight findings were identified and remediated, the highest-severity classes (command execution, SSRF, local-API exposure, loopback rebinding, parser resource-exhaustion) closed with regression coverage. The local-first architecture removes the largest class of risk by

construction — there is no vendor-side system holding customer documents to breach. Hardening continues under the roadmap above.
